4 Roteiro 04 – Integração Flask com Interface Web

Objetivos

- Integrar backend Flask com formulário HTML;
- Compreender Template Engine (Jinja2);
- Trabalhar métodos GET e POST;
- Manipular dados em memória;
- Manter API JSON funcionando;
- Aplicar versionamento organizado.

Ferramentas Necessárias

Python 3.11+, Flask, Git.

Introdução

Nas aulas anteriores criamos uma API básica utilizando Flask e retornando dados em formato JSON.

Agora evoluiremos para integrar:

- Interface HTML;
- Formulário web;
- Backend Flask;
- API REST simultaneamente.

O objetivo é compreender como frontend e backend se comunicam em uma aplicação real.

Fundamentação Teórica

Conceito	Explicação	Exemplo no Projeto
Template Engine	Sistema que permite inserir dados dinâmicos dentro do HTML.	`{% for aluno in alunos %}`
Renderização no Servidor	O HTML é montado no backend antes de ser enviado ao navegador.	`return render_template("index.html", alunos=alunos)`
Formulário HTML	Estrutura que permite enviar dados do navegador para o servidor.	`<form method="POST»`
request.form	Captura dados enviados pelo formulário.	`nome = request.form["nome"]`
redirect	Redireciona o navegador após uma ação.	`return redirect("/")`
Estado em Memória	Dados temporários armazenados em lista.	`alunos = []`
GET vs POST	GET recupera dados; POST envia dados.	`@app.route("/", methods=["GET"])`
API JSON	Retorna dados estruturados.	`return jsonify(...)`

Estrutura do Projeto

```
api-alunos/

|-- aluno.py
|-- app.py
|-- templates/
|   |-- index.html
```

Resultado Final Esperado

Ao final deste roteiro, você será capaz de construir uma aplicação web simples com:

- Formulário para cadastro de alunos;
- Integração entre HTML e backend Flask;
- Atualização dinâmica da lista de alunos;
- API JSON funcionando simultaneamente.

Abaixo está uma representação da interface que será desenvolvida:

O que observar na interface:

- O usuário pode inserir nome e idade;
- Ao enviar, os dados são processados pelo backend;
- A lista de alunos é atualizada automaticamente;
- Não há recarregamento manual — o sistema controla o fluxo.

Parte 1 – Criando a Pasta Templates

```
mkdir templates
touch templates/index.html
```

Explicação:
O Flask utiliza uma convenção onde todos os arquivos HTML devem estar dentro da pasta `templates`. Quando utilizarmos `render_template()`, o Flask irá procurar automaticamente o HTML nesta pasta. Neste momento ainda não há troca de dados, mas estamos preparando o ambiente para que o backend consiga enviar informações ao frontend.

Parte 2 – Ajustando o app.py

```
from flask import Flask, jsonify, request, render_template, redirect
from aluno import Aluno

app = Flask(__name__)

alunos = []

@app.route("/", methods=["GET"])
def pagina_inicial():
    return render_template("index.html", alunos=alunos)

@app.route("/alunos", methods=["GET"])
def listar_alunos():
    return jsonify([aluno.to_dict() for aluno in alunos])

@app.route("/alunos", methods=["POST"])
def adicionar_aluno():
    nome = request.form["nome"]
    idade = int(request.form["idade"])

    novo_aluno = Aluno(nome, idade)
    alunos.append(novo_aluno)

    return redirect("/")

if __name__ == "__main__":
    app.run(debug=True)
```

Explicação:
Aqui criamos três fluxos principais:

- GET "/"→ envia dados para o HTML;
- GET "/alunos"→ retorna JSON;
- POST "/alunos"→ recebe dados do formulário.

Fluxo do POST:

1. Usuário envia formulário;
2. Dados chegam via `request.form`;
3. Criamos objeto Aluno;
4. Salvamos na lista;
5. Redirecionamos para "/";

Parte 3 – Criando o HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Cadastro de Alunos</title>
</head>
<body>

<h1>Cadastro de Alunos</h1>

<form method="POST" action="/alunos">
    Nome:
    <input type="text" name="nome" required>

    Idade:
    <input type="number" name="idade" required>

    <button type="submit">Cadastrar</button>
</form>

<hr>

<h2>Lista de Alunos</h2>

<ul>
{% for aluno in alunos %}
    <li>
        {{ aluno.nome }} - {{ aluno.idade }} anos
    </li>
{% endfor %}
</ul>

</body>
</html>
```

Explicação:
O HTML possui duas funções:

- Enviar dados (formulário);
- Exibir dados (lista).

Fluxo:

1. Backend envia lista de alunos;
2. Jinja2 renderiza os dados;
3. Usuário envia novo aluno;
4. Dados voltam ao servidor.

Parte 4 – Executando a Aplicação

```
python app.py
```

Acesse:

`http://127.0.0.1:5000`

Teste também:

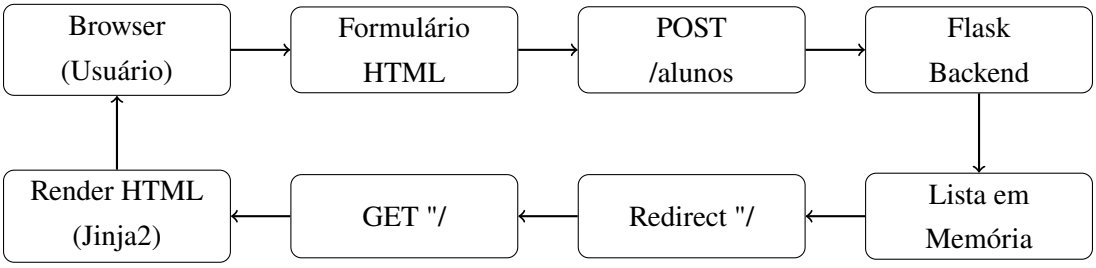
`http://127.0.0.1:5000/alunos`

Explicação:
Agora conseguimos observar dois tipos de resposta:

- HTML (interface visual);
- JSON (dados estruturados).

A mesma informação é usada em dois formatos diferentes.

Fluxo Completo da Aplicação



Leitura do fluxo:

1. Usuário envia dados pelo formulário;
2. Backend processa e salva;
3. Redireciona;
4. Página é reconstruída com novos dados.

Consolidação dos Conceitos

Resumo dos Conceitos Utilizados

Conceito	O que é	O que acontece no sistema
Jinja2 (Template Engine)	Permite inserir dados dinâmicos dentro do HTML.	O Flask envia dados e o HTML é montado dinamicamente com base nesses dados.
Renderização no Servidor	O backend monta o HTML antes de enviar ao navegador.	O usuário recebe a página pronta, já com os dados inseridos.
Formulário HTML (POST)	Estrutura para envio de dados do usuário para o servidor.	Os dados digitados são enviados para o backend via requisição POST.
request.form	Objeto do Flask que captura dados enviados pelo formulário.	O backend acessa os valores enviados pelo usuário e processa esses dados.
Estado em Memória	Armazenamento temporário usando estruturas Python (lista).	Os dados ficam disponíveis enquanto o servidor está rodando e são perdidos ao reiniciar.
API JSON	Interface que retorna dados estruturados.	Os mesmos dados podem ser consumidos por outras aplicações ou sistemas.
GET vs POST	Métodos HTTP com finalidades diferentes.	GET busca dados; POST envia dados para processamento no servidor.

Testando Requisições POST

Por que ficou mais difícil testar o POST?

Neste roteiro, o backend foi configurado para receber dados a partir de formulários HTML utilizando:

```
request.form["nome"]
```

Isso significa que o servidor espera os dados no formato:

- `application/x-www-form-urlencoded`

Ferramentas como Postman e cURL, por padrão, costumam trabalhar com JSON, o que causa incompatibilidade.

Por que o formulário funciona automaticamente?

O navegador já envia os dados no formato correto ao utilizar um formulário HTML:

- Método POST;
- Dados enviados no corpo da requisição;
- Formato compatível com `request.form`.

Por isso, o Flask recebe os dados corretamente sem configuração adicional.

Como testar com Postman?

Para funcionar corretamente no Postman:

- Método: POST
- URL: `http://127.0.0.1:5000/alunos`
- Body → selecionar **x-www-form-urlencoded**

Campos:

```
nome: Maria
idade: 22
```

Como testar com cURL?

```
curl -X POST http://127.0.0.1:5000/alunos \
-d "nome=Maria" \
-d "idade=22"
```

O uso do `-d` envia os dados no formato esperado pelo Flask.

E se quisermos usar JSON novamente?

Seria necessário alterar o backend:

```
data = request.get_json()

nome = data["nome"]
idade = data["idade"]
```

Nesse caso, o cliente (Postman ou frontend) também deve enviar JSON.

Resumo

- Formulário → envia dados automaticamente no formato correto;
- Postman/cURL → exigem configuração manual;
- JSON e form-data são formatos diferentes;
- O backend precisa estar preparado para o formato utilizado.

Versionamento

```
git add .
git commit -m "Integra formulário HTML com Flask mantendo API JSON"
```

Entrega

- Aplicação funcionando;
- Formulário com POST;
- Lista dinâmica;
- API JSON ativa;
- Histórico Git.